Chess - programming documentation

Author: Lukáš Hofman

Minimax Class

Overview

The Minimax class is used to implement the minimax algorithm with alpha beta pruning for making optimal movesin the game of chess. It provides methods for finding the best moves for the current player and generating responses to opponent moves.

It uses Tasks to do the computations asynchronously to be able to compute before a move is made by the player and to be slightly faster when the player does make a move.

It uses alpha beta pruning to increase the computational power of the minimax algorithm.

Constructors

- `Minimax(ChessUI currentBoard, int level = 0)`: Initializes a new instance of the Minimax class with the specified ChessUI instance representing the current game board and an optional parameter `level` indicating the depth of the minimax algorithm and starts the computations.

Methods

- `public void Start(ChessUI currentBoard)`: Starts the minimax algorithm, computing the best responses to the current board state asynchronously.
- `pubic void StartOver(ChessUI currentBoard)`: Cancels the previous computations and restarts the minimax algorithm from scratch, computing the best responses to the current board state asynchronously.
- `public void MinimaxPlay(ChessUI currentBoard, Move lastMove)`: Plays the best move for the current player based on the current board state and the last move made by the opponent. If a best response to the opponent's move has been computed previously, it is played immediately; otherwise, the minimax algorithm is used to compute the best response asynchronously.
- `public void Cancel()`: Cancels the computations of the minimax algorithm and waits for them to complete.
- `private static double ScorePiece(Piece piece)`: Evaluates the Value of the piece depending on the position of the board and state of

the game.

- `private static double Evaluate(ChessUI currentBoard)`: Evaluates the current board state and returns a score indicating the desirability of the position for the current player. It uses a random to make the games a little more diverse.
- `private static double MinimaxValue(ChessUI currentBoard, int depth, CancellationToken cancellationToken, double alpha = double.MinValue, double beta = double.MaxValue)`: Computes the value of the current board state using the minimax algorithm with alpha-beta pruning. The `depth` parameter specifies the depth of the search, and the `cancellationToken` allows cancellation of the computation.
- `private static async Task<Move> BestPlay(ChessUI currentBoard, int depth, CancellationToken cancellationToken)`: Finds the best move for the current player using the MinimaxValue function to evaluate the state of the game after all possible moves. The `depth` parameter specifies the depth of the search, and the `cancellationToken` allows cancellation of the computation.
- `private static List<Move> MovesOrderedByQuality(ChessUI currentBoard, int depth, CancellationToken cancellationToken)`: Generates an estimated ordered list of moves from best to worst based on the current board state. The `depth` parameter specifies the depth of the search, and the `cancellationToken` allows cancellation of the computation.
- `public async void FindBestResponses(ChessUI currentBoard, int level, CancellationToken cancellationToken)`: Finds the best response to the current board state for each possible move of the current player. The `level` parameter specifies the depth of the minimax search, and the `cancellationToken` allows cancellation of the computation.

Fields

- `private ConcurrentDictionary<Move, Move> _bestResponses`: A dictionary storing the best responses computed for each possible move of the opponent.
- `private Task _minimax`: A task running the async void functions
- `private CancellationTokenSource _cancellationTokenSource`: A CancellationTokenSource used for canceling the computations of the minimax algorithm.

Properties

- `private int _level`: The max depth the minimax search algorithm is going to go. - the level of the AI opponent

Chess Form Class

Overview

The `Chess` class represents the main form of the chess game application. It handles the graphical user interface and combines the non-graphical user interface and game logic with the minimax algorithm.

Fields

- `private bool vsAI`: Indicates whether the game is played against the AI.
- `private ChessUI? game`: Represents the current chess game.
- `private Position? previousClick`: Stores the position of the previously clicked panel.
- `private Position? currentClickCoordinates`: Stores the coordinates of the currently clicked panel.
- `private Panel[,] panels`: Stores the graphical panels representing the chessboard.
- `private Label[] boardLabels`: Stores the labels representing the row and column indicators of the chessboard.
- `private List<PictureBox> pieces`: Stores the graphical representations of chess pieces.
- `private Button[] promotionButtons`: Stores the promotion buttons for pawn promotion.
- `private bool cellsCreated`: Indicates whether the chessboard panels have been created.
- `private Pawn? promotionPiece`: Stores the pawn to be promoted during pawn promotion.
- `private List<Move> highlightedMoves`: Stores the highlighted moves for visual indication.
- `private Label gameendl`: Represents the label indicating the end of the game.
- `private List<string> history`: Stores the FEN strings of previous game states for undo functionality.
- `private string lastFEN`: Stores the FEN string of the last game state.
- `private Minimax minimax`: Manages the minimax algorithm for AI moves.
- `private bool playAsWhite`: Indicates whether the player plays as white. (Only relevant if the player wants to play against minimax)

Constructors

- `public Chess()`: Initializes a new instance of the Chess class.

Methods

- `public void CreateCells()`: Creates the graphical panels representing the chessboard and adds event handlers for click events.
- `public void StartGame()`: Initializes the game by creating the graphical representation of pieces on the board.
- `private void ColorPossibleMoves(Position position, bool back = false)`: Highlights or removes highlights from panels indicating possible moves.
- `private void ColorKingCheck()`: Highlights the king or removes highlight from it if the king is in check.
- `private void ColorPlayedMove()`: Highlights the played move and removes the highlight of the previously played move.
- `private void UpdateGameBoard()`:Updates the graphical representation of the chessboard based on the current game state.
- `private void UpdateGame()`: Updates the graphical representation after a move is made, if player plays against AI, a move by minimax is played and game updates accordingly.
- `private Position? GetPositionOfClick(int x, int y)`: Converts mouse click coordinates to panel coordinates.
- `private void Cell_Click(object? sender, EventArgs e)`: Handles click events on the panels, allowing the player to make moves.
- `private void SwitchInterface(bool toGame = true)`: Switches between the main menu and game interface, hides, disables or reveals and enables all necessary components.
- `private void CreateEndGameLabel()`: Creates a label indicating the winner of the game and adds it to the form.
- `private void playb_Click(object sender, EventArgs e)`: Starts the game by creating a new `ChessUI` object and calling the `StartGame` function. It also hides the start menu.
- `private void exitb_Click(object sender, EventArgs e)`: Closes the application.
- `private void vsAIb_Click(object sender, EventArgs e)`: Toggles between player vs. player and player vs. AI mode, and adjusts UI elements accordingly.

- `private void AIdifficultybar_Scroll(object sender, EventArgs e)`: Updates the displayed AI difficulty level based on the scroll bar value.
- `private void playasb_Click(object sender, EventArgs e)`: Toggles between playing as white and playing as black.
- `private void saveb_Click(object sender, EventArgs e)`: Opens a file dialog and saves the game to the selected file.
- `private void loadb_Click(object sender, EventArgs e)`: Opens a file dialog and loads the game from the selected file.
- `private void ResetColorsOfPlayedMove()`: Resets the colors of the played move.
- `private void backb_Click(object? sender, EventArgs e)`: Undoes the last move and updates the game board.
- `private void mainmenub_Click(object sender, EventArgs e)`: Switches from the game interface to the main menu.
- `private void CreatePromotionButtons()`: Creates buttons for pawn promotion and adds them to the form.
- `private void PromotionUpdate()`: Updates the game board and removes promotion buttons after a pawn is promoted.
- `private void Queen_Click(object? sender, EventArgs e)`: Handles the promotion of a pawn to a queen.
- `private void Rook_Click(object? sender, EventArgs e)`: Handles the promotion of a pawn to a rook.
- `private void Bishop_Click(object? sender, EventArgs e)`: Handles the promotion of a pawn to a bishop.
- `private void Knight_Click(object? sender, EventArgs e)`: Handles the promotion of a pawn to a knight.

ChessUI

Enumerations

StateOfGame

An enumeration describing the state of the game. Possible values are:

- `Game`: The game is ongoing.
- `Check`: One player is in check.
- `PromotionPending`: A pawn is eligible for promotion.
- `Draw`: The game ended in a draw.
- `WhiteWins`: White player has won the game.

- `BlackWins`: Black player has won the game.

Structures

Position

A structure representing a position on the chessboard. It consists of two integers `x` and `y` representing the coordinates.

Move

A structure representing a move in the game. It consists of two `Position` objects: `from` and `to`, indicating the start and end positions of the move.

Classes

Cell

A class representing a cell on the chessboard. It contains information regarding the cell.

Fields

- `public Piece? piece`: List of all white pieces.
- `public List<Piece> attackedByWhite`: List of all white pieces attacking the Cell.
- `public List<Piece> attackedByBlack`: List of all black pieces attacking the Cell.

ChessUI

Overview

The `ChessUI` class is the main class representing the state of a chess game. It manages the board, pieces, game state, and game logic.

Fields

- `whitePieces`: List of all white pieces.
- `blackPieces`: List of all black pieces.
- `possibleMoves`: List of all possible moves at the current state.
- `stopKingCheck`: List of positions that stop the king from being in check.
- `board`: 2D array representing the board.
- `kings`: Array containing the white and black kings.

Properties

- `CastlingRights`: Array representing the castling rights.
- `EnPassant`: Position of the en passant square.
- `State`: Current state of the game.
- `HalfMoves`: Half-move clock for the fifty-move rule.
- `FullMoves`: Number of full moves.
- `WhiteToMove`: Boolean indicating whether it's White's turn to move.
- `MinimaxActive`: Boolean indicating whether the UI is for minimax.

Constructor

- `ChessUI(bool minimax = false)`: Creates a brand new chess game.
- `ChessUI(StreamReader sr, bool minimax = false)`: Creates a new chess game from a file.
- `ChessUI(string FEN, bool minimax = false)`: Creates a new chess game from a FEN string.
- `ChessUI(ChessUI ui, bool minimax = true)`: Creates a deep copy of an existing game. (minimax is set to true by default because the minimax algorithm uses this function alot)

Methods

- `void SaveGame(string fileName)`: Creates/opens a file and saves the game there.
- `void SaveGame(StreamWriter file)`: Saves the game to a file.
- `static ChessUI LoadGame(string fileName)`: Loads a game from a file.
- `void Check()`: Checks if the current player is in check and if it is, changes the State of the game to Check.
- `void GameEnd()`: Checks if the game is over and if it is - changes the State of the game to the corresponding state.
- `bool Move(string from, string to)`: Moves a piece from one position to another (using string coordinates).
- `bool Move(Position from, Position to)`: Moves a piece from one position to another.
- `bool Move(Move move)`: Moves a piece based on a `Move` object.
- `string GetFEN()`: Returns the string FEN representation of the current game.
- `void PrintBoard()`: Prints the current board state to the console. (This function is obsolete, used only on ConsoleChess)

Private Methods

- `void UpdateMovesOfAllPieces()`: Updates possible moves for all pieces. Kings have to be updated last because their possible moves depend on other pieces.

- `void FillPieceAttacks()`: Fills the attackedByWhite and attackedBy-Black lists in each cell by Updating moves of all pieces and corrects the state of the game.
- `bool NotEnoughPieces()`: Checks if there are enough pieces for either player to win.
- `void DeepCopyBoardToThisGame(Cell[,] board)`: Creates a deep copy of the board.
- `Cell[,] CreateBoardFromFen(string fen)`: Creates a board from a FEN string.

Usage without the Winforms (Promotion of Pawns is not updated for Console usage)

```
// Example of creating a new chess game
ChessUI game = new ChessUI();

// Example of printing the board
game.PrintBoard();
// Example of making a move
game.Move("e2", "e4");
game.PrintBoard();

// Example of saving the game to a file
game.SaveGame("saved_game.txt");

// Example of loading a game from a file
ChessUI loadedGame = ChessUI.LoadGame("saved_game.txt");

game.PrintBoard();
```

if you want to use the code for ConsoleChess, you will have to udpate the Pawn Move function a little for it to work properly - here is a suggestion how you could change the function to make it work on Console:

instead of this part of the Pawn Move function:

```
// checks if the position the pawn moved is on the edge of the board - promotion needed
if (where.y == 0 || where.y == 7)
{
    if (this.UI.MinimaxActive)
    {
        Promote(PromotePieceType.Queen, where);
    }
    else
    {
        this.UI.State = StateOfGame.PromotionPending;
    }
```

```
}
else
{
    UpdateMoveAndPosition(where);
}
```

use this instead

```
// checks if the position the pawn moved is on the edge of the board - promotion needed
if (where.y == 0 || where.y == 7)
{
    UI.state_ = StateOfGame.PromotionPending;
    bool correctPromotion = false;
    PromotionUpdateOfMoves(where);
    while (!correctPromotion)
    {
        Console.Write("Promote to: ");
        string piece_s = Console.ReadLine();
        int piece = piece_s[0];
        correctPromotion = true;
        switch (piece)
        {
            case 'q':
            case 'Q':
                Promote(PromotePieceType.Queen);
                break;
            case 'r':
            case 'R':
                Promote(PromotePieceType.Rook);
                break;
            case 'b':
            case 'B':
                Promote(PromotePieceType.Bishop);
                break;
            case 'n':
            case 'N':
                Promote(PromotePieceType.Knight);
                break;
            default:
                correctPromotion = false;
                break;
        }
    }
}
else
{
    UpdateMoveAndPosition(where);
```

}

Piece

Enumerations

PromotePieceType An enumeration representing the types of pieces that a pawn can be promoted to:

- `Queen`: Promote to a queen.
- `Rook`: Promote to a rook.
- `Bishop`: Promote to a bishop.
- `Knight`: Promote to a knight.

Classes

Piece An abstract class representing a chess piece. It contains properties and methods common to all chess pieces. All pieces inherit from this class.

It serves as a base for implementing specific chess pieces like pawn, knight, bishop, rook, queen, and king.

Properties

- `ChessUI UI`: Reference to the `ChessUI` object to which the piece belongs.
- `Position position`: The position of the piece on the chessboard.
- `bool white`: Boolean indicating whether the piece is white (`true`) or black (`false`).
- `List<Position> blocking_`: List of positions where a piece could potentially move but is unable to (a piece blocking the way, no enemy piece near a Pawn,...)
- `List<Position> possibleMoves`: List of positions where the piece can move to.

Methods

- `string image()`: Abstract method to return the path to the image of the piece.
- `void UpdatePosibleMoves()`: Abstract method to update the possible moves of the piece.
- `bool MoveLeavesKingInCheck(Position where)`: Checks if a move leaves the king in check.
- `bool CheckMove(Position where)`: Checks if a move is valid.
- `void TakingOfAPiece(Position where)`: Handles taking of a piece from the board.
- `void Move(Position where)`: Moves the piece to the given position.

- `void CellsStoppingCheck(List<Position> highlightedCells)`: Abstract method to fill the highlightedCells list with the positions of cells that the piece is blocking.
- `void RemoveAll()`: Clears all moves, attacks, and blockings of the piece.
- `void UpdateMoveAndPosition(Position where)`: Updates the position of the board, possible moves of affected pieces, switches the current player, and updates the number of moves.
- `void UpdatePawnMoves()`: Updates the moves of pawns that were previously blocked.
- `void UpdateMovesOfCellAttackers(Cell cell)`: Updates the moves of attackers of a cell.
- `void AddBishopMoves()`: Adds the moves of a bishop to possible moves of the piece.
- `void AddRookMoves()`: Adds the moves of a rook to possible moves of the piece.

Pawn Class

Brief

Class representing the Pawn piece.

Constructor

- `Pawn(Position position, bool white, ChessUI UI)`: Initializes a Pawn object with the given position, color, and ChessUI reference.

Properties

- `Value` Returns the value of the Pawn piece, which is 1.

Methods

- `image()`: Returns the image path of the Pawn piece based on its color.
- `CellsStoppingCheck(List<Position> highlightedCells)`: Adds the current position of the Pawn to the list of highlighted cells.
- `UpdatePosibleMoves()`: Updates the list of possible moves for the Pawn. It adds En-passant movement and handles the initial two cell move.
- `Move(Position where)`: Moves the Pawn to the specified position.

Additional Methods

- `PromotionUpdateOfMoves(Position where)`: Updates the possible moves of pieces after promotion. (instead of normal update, functions are very similar)
- `Promote(PromotePieceType promotion, Position where)`: Promotes the Pawn to a selected piece.

King Class

Brief

Class representing the King piece.

Constructor

- `King(Position position, bool white, ChessUI UI)`: Initializes a King object with the given position, color, and ChessUI reference.

Properties

- `Value`: Returns the value of the King piece, which is 0. King has to be always on the board so his Value is not important.

Methods

- `image()`: Returns the image path of the King piece based on its color.
- `CellsStoppingCheck(List<Position> highlightedCells)`: Does nothing as King can't block himself from a check.
- `UpdatePosibleMoves()`: Updates the list of possible moves for the King. Handles castling.
- `Move(Position where)`: Moves the King to the specified position.

Queen Class

Brief

Class representing the Queen piece.

Constructor

- `Queen(Position position, bool white, ChessUI UI)`: Initializes a Queen object with the given position, color, and ChessUI reference.

Properties

- `Value`: Returns the value of the Queen piece, which is 9.

Methods

- `image()`: Returns the image path of the Queen piece based on its color.
- `CellsStoppingCheck(List<Position> highlightedCells)`: Adds the positions stopping the check by the Queen to the list of highlighted cells.
- `UpdatePosibleMoves()`: Updates the list of possible moves for the Queen. (basically Adds bishop and rook moves)

Bishop Class

Brief

Class representing the Bishop piece.

Constructor

- `Bishop(Position position, bool white, ChessUI UI)`: Initializes a Bishop object with the given position, color, and ChessUI reference.

Properties

- `Value`: Returns the value of the Bishop piece, which is 3.

Methods

- `image()`: Returns the image path of the Bishop piece based on its color.
- `CellsStoppingCheck(List<Position> highlightedCells)`: Adds the positions stopping the check by the Bishop to the list of highlighted cells.
- `UpdatePosibleMoves()`: Updates the list of possible moves for the Bishop.

Knight Class

Brief

Class representing the Knight piece.

Constructor

- `Knight(Position position, bool white, ChessUI UI)`: Initializes a Knight object with the given position, color, and ChessUI reference.

Properties

- `Value`: Returns the value of the Knight piece, which is 3.

Methods

- `image()`: Returns the image path of the Knight piece based on its color.
- `CellsStoppingCheck(List<Position> highlightedCells)`: Adds the position of the Knight to the list of highlighted cells.
- `UpdatePosibleMoves()`: Updates the list of possible moves for the Knight.

Rook Class

Brief

Class representing the Rook piece.

Constructor

- `Rook(Position position, bool white, ChessUI UI)`: Initializes a Rook object with the given position, color, and ChessUI reference.

Properties

- `Value`: Returns the value of the Rook piece, which is 5.

Methods

- `image()`: Returns the image path of the Rook piece based on its color.
- `CellsStoppingCheck(List<Position> highlightedCells)`: Adds the positions stopping the check by the Rook to the list of highlighted cells.
- `UpdatePosibleMoves()`: Updates the list of possible moves for the Rook.
- `Move(Position where)`: Moves the Rook to the specified position.